Residual Defects and Temporal-Evaluation Pitfalls in the Corrected CIC-IDS-2017 and CSE-CIC-IDS-2018 Datasets

Vyom Khanna
B.Tech., Thapar Institute of Engineering and Technology, Patiala, Punjab, India

***Abstract*—The corrected releases of CIC-IDS-2017 and CSE-CIC-IDS-2018 published by Liu, Engelen et al. at IEEE CNS 2022 fixed a long list of flow-construction and labelling errors, and they are now being adopted as drop-in replacements for the originals. Their microsecond UTC timestamps also make them attractive for temporal work such as next-window attack prediction. We report a full-scan audit of the post-publication versions of both corrected releases (65.3 million flows) against the original releases (19.1 million data rows). The labels agree with the released labelling logic for all 4,359,069 attack and attempted flows, but we find defects that the corrections introduced or left in place: the forward and backward header-length features were cast to 16-bit integers and overflow, 2,609 flows in the 23 February 2018 file are dated a day before the capture, and feature-identical rows still make up 12.5% and 28.5% of the two releases. We also show that both original releases use a 12-hour clock without an AM/PM marker. For temporal use, attacks form only 24 and 38 scripted episodes, persistence of the current window explains 67–73% and 91–93% of next-window positives, a quarter to a third of flows outlast a 2-second window, and random-row splits place exact feature twins of 15.6% and 30.0% of test rows in training. We release the audit code and tables and propose an evaluation protocol that makes these effects visible.**

***Index Terms*—network intrusion detection, datasets, data quality, CIC-IDS-2017, CSE-CIC-IDS-2018, data leakage, temporal evaluation.**

I. Introduction

Machine-learning network intrusion detection (NIDS) is still evaluated mostly on public benchmark datasets, and for several years CIC-IDS-2017 [1] and CSE-CIC-IDS-2018 have been the most common choices. Both were produced with the same pipeline: scripted attacks in a testbed, packet captures, flow records exported by CICFlowMeter, and labels assigned afterwards from attack schedules. Each step has since been shown to contain errors. Engelen et al. [2] traced problems in CIC-IDS-2017 to flow termination and labelling; Rosay et al. [3] and Lanvin et al. [4] reported further flow-extraction and labelling faults; and Liu, Engelen et al. [5] (the CNS 2022 paper that this work builds on) documented labelling error rates of 6.67% and 7.53% in the two datasets, fixed the extractor, regenerated the flows from the captures, and released corrected datasets together with their labelling code.

A corrected dataset is not an error-free one, and the corrections themselves are code that can fail. The CNS 2022 authors say as much: they expected residual problems and published fixes after the paper appeared. Downstream users rarely check. They load the corrected CSV files, assume that the documented problems are gone, and in many cases change the task at the same time, for instance from per-flow classification to forecasting future attack activity from a time series of network state. That new task depends on properties of the data that the original audits did not examine: the timestamp semantics, the ordering of rows, how attacks are distributed in time, and how flow records that span many seconds interact with fixed time windows.

This paper asks two questions. First, what defects remain in the corrected releases, and how do the corrected and original releases differ when compared at full scale? Second, which properties of the corrected data decide whether a temporal evaluation on them is meaningful? We answer both with full scans of all 33 files of the four releases and with an independent recomputation of the headline numbers. No model is trained; every result in this paper is a count over the data.

Our contributions are:

- a defect inventory of the post-publication corrected releases, including a 16-bit overflow in two header-length features, a block of stray flows dated a day before the capture, and the label composition of the remaining feature-identical conflict groups (Section IV);
- evidence that both original releases record local time on a 12-hour clock without AM/PM, at UTC−3 (2017) and UTC−4 (2018), and a measurement of how far row-level matching between releases is possible (Section V);
- a quantitative account of the temporal structure of the corrected data (episode counts, persistence, look-ahead from long flows, split leakage and single-feature shortcuts) that bounds what a forecasting result on these data can mean (Section VI);
- an evaluation protocol derived from these measurements, with the audit code and all result tables released (Section VII).

II. Background and Related Work

A. The datasets and their corrections

CIC-IDS-2017 [1] covers five working days (3–7 July 2017) of traffic in a small enterprise testbed, with brute-force, DoS, DDoS, web, infiltration, botnet and port-scan attacks. CSE-CIC-IDS-2018 repeats the design on AWS infrastructure over ten days in February and March 2018. Both were distributed as

CICFlowMeter CSV files with 80–85 columns. Engelen et al. [2] showed that the original extractor terminated TCP flows at the first FIN packet, so that the remainder of each connection became a second, short "TCP appendix" flow, and that several attacks were labelled with incorrect time windows. The CNS 2022 work [5] extended this analysis to CSE-CIC-IDS-2018, corrected the extractor (FIN/RST handling, bulk features, segmentation offload, ICMP, timestamps in UTC with microseconds), regenerated both datasets, and introduced an "Attempted" label suffix for flows that belong to an attack but carry no malicious payload. The authors published post-publication fixes in March and April 2023. An independent corrected version, LycoS-IDS2017, was produced with a different extractor [3], so the phrase "the corrected CIC-IDS-2017" is ambiguous unless the release is named.

B. Evaluation pitfalls

Arp et al. [6] list data snooping, including temporal snooping, and spurious correlations among the most frequent pitfalls in security machine learning. Kaufman et al. [7] give a general account of leakage, and Geirhos et al. [8] describe shortcut learning. For NIDS specifically, D'hooge et al. [9] show that the destination port alone separates the classes of many datasets, and Cantone et al. [10] find that models trained on one of CIC-IDS-2017, CSE-CIC-IDS-2018 and their LycoS variants transfer poorly to the others. Flood et al. [11] catalogue recurring design flaws in NIDS benchmarks. Moczkodan and Ragab [12] report that random splits and overlapping windows inflate sequence-model results on the original CIC-IDS-2017, and Youssef [13] shows that the way a stream is assembled from captures can change model rankings.

C. Prediction and forecasting

Husák et al. [14] separate attack projection, attack prediction and situation forecasting. Ahmed et al. [15] forecast event series derived from the original CSE-CIC-IDS2018, and a recent preprint, SENTINEL [16], forecasts compromise up to 20 s ahead from windows of behavioural features on the 2018 infiltration scenario. Other work predicts the current attack stage from alert windows [17] or classifies flows from their first packets, which is earliness within a flow rather than forecasting. We located 421 unique works that cite either the CNS 2022 paper or Engelen et al. [2] through OpenAlex and Semantic Scholar and screened them by title and abstract. None of those we could read used the corrected CSE-CIC-IDS-2018 for a temporal task, although an audit framework, IDS2Eval [18], includes the corrected CIC-IDS-2017. We therefore do not claim that forecasting on these data is new; our concern is what the data allow such a result to mean.

III. Data and Method

A. Releases

Table I lists the four releases. The corrected archives were downloaded from the authors' distribution site; their sizes match the server and every archive member passed its CRC check. The archive timestamps coincide with the post-publication fix commits in the authors' labelling repository (30 March to 27 April 2023), so these are the corrected files *as currently distributed*, not necessarily the files behind the tables in [5]. The 2018 originals are byte-identical to the objects in the public AWS bucket: all ten recomputed multipart ETags match. No official checksum exists for the 2017 originals, but their non-empty row count equals the figure published in [5].

TABLE I
Releases audited (rows read by full scan)

Release	Files	Data rows	Time stamp
2017 original	8	2,830,743[a]	local, minutes
2017 corrected	5	2,099,976	UTC, µs
2018 original	10	16,232,943[b]	local, seconds
2018 corrected	10	63,195,145	UTC, µs

[a]Plus 288,602 all-empty rows. [b]Plus 59 repeated header rows; 7 of 10 files hold exactly 1,048,575 rows.

B. Audit procedure

Every statistic was computed over all rows, not samples. Files were read with DuckDB 1.5.6 [19] with every column declared as text and parsed explicitly, and each row count was checked against a byte-level count of line terminators. This check caught one original file whose line endings change from CRLF to LF part-way through; a reader configured for one terminator silently skipped about 600,000 of its rows. Two files that could not be read as they stand (one with Windows-1252 bytes, the one with mixed line endings) were transcoded into a temporary cache, never in place, and every cached copy is hash-checked against a manifest.

Duplicates are counted as copies beyond the first, over data rows only. A pair of rows is *feature-identical* when it agrees on all feature and context columns after removing the row index, flow identifier, addresses, source port, timestamp and labels; numeric values are compared as numbers, because the files mix forms such as 0 and 0.0. For a group of feature-identical rows with more than one label, the number of rows outside the majority label is a lower bound on the errors of any classifier that sees only these features; we call the sum over groups the *conflict floor*. Single-feature separability is measured by the overlap coefficient (OVL) of the two class-conditional histograms on $\log_2$ bins, where 0 means that one feature separates the classes perfectly and 1 means that it carries no information.

To guard against errors in our own pipeline, the headline numbers were recomputed by a second program that shares no reader, query or cache with the first (pyarrow and pandas, streaming the raw files once). All 17 checks agreed. Separately, 32 numbers quoted in our reports were regenerated from the result tables and matched against the text. Several numbers that we had stated earlier in the work were wrong and are corrected here; the most important was a 2017 duplicate share that had counted empty rows as duplicates of each other.

IV. Defects in the Corrected Releases

A. Labels agree with the released logic

Each attack in the corrected releases is labelled by a rule that combines a time window with filters on addresses, ports and

flow statistics. We extracted all documented windows from the labelling notebooks and checked every non-benign flow against them. All 4,359,069 attack and Attempted flows (517,410 in 2017 and 3,841,659 in 2018) start inside a window for the same label, Attempted category and day; none falls outside and no label lacks a rule. The check runs in one direction only: it shows that every attack label is supported, not that every flow matching a rule was labelled.

B. Header-length overflow

In both corrected releases, every value of *Fwd Header Length* and *Bwd Header Length* lies within [−32,768, 32,767], the range of a signed 16-bit integer. Negative values occur in 309 and 472 rows of 2017 and in 10,039 and 17,509 rows of 2018. The same features reach 4,644,908 and 5,838,440 bytes in the 2017 originals. The labelling notebooks cast these columns with astype('int16') before export, so any header total above 32,767 bytes wrapped around. The negative values are a lower bound on the damage: a sufficiently large total wraps to a positive value, and indeed 152 (2017) and 3,951 (2018) TCP flows have a positive forward header length below 20 bytes per packet, which is impossible for real TCP headers. We recommend excluding both features or recomputing them from the captures. The other narrowed columns (ICMP code and type, flow duration) stay within their ranges.

C. Changed sentinels and residual values

The initial-window-bytes features use −1 for "not observed" in the originals (about 1.0 and 1.4 million rows in 2017, 4.4 and 8.3 million in 2018). The corrected releases contain no −1 at all, while the share of zeros rises to 40–61%, which suggests that "not observed" is now encoded as 0. Raw values of these features are therefore not comparable across releases. Non-finite values of *Flow Bytes/s* fall from 2,867 to 5 (2017) and from 95,760 to 57 (2018); the remaining few must still be handled. The negative durations and inter-arrival times of the originals are gone. One extractor bug survives in a weaker form: in the 2018 corrected release, *URG Flag Count* equals *Fwd URG Flags* in every row.

D. Stray flows before the capture

The file for 23 February 2018 contains 2,609 benign flows timestamped between 21 February 12:33 and 22 February 12:21 UTC, roughly a day before the capture it belongs to. They consist of sparse traffic with external addresses and leave a 23.9-hour gap in the file. The authors documented and removed a similar block of stray capture data for 2 March 2018; this one remains. Apart from it, no corrected day has a gap longer than 130 s between consecutive flow starts. The stray rows matter for temporal work: left in, they stretch the time grid of that day by about 86,000 empty 2-second windows, which in our first computation lowered the 2018 next-window positive rate from 10.35% to 7.39%, and they shift chronological split points.

E. Duplicates and label conflicts

Table II compares duplicates across the four releases. The corrected releases contain no exact duplicates, which is expected because every row carries a microsecond timestamp and a 5-tuple. Once identifiers are removed, however, 12.52% and 28.47% of their rows are copies of another row, slightly more than in the originals. The corrections did not make flows more distinct.

TABLE II
DUPLICATES AND LABEL CONFLICTS (DATA ROWS ONLY)

	2017 orig.	2017 corr.	2018 orig.	2018 corr.
Exact duplicates	203	0	433,195	0
Feature-identical	308,848	263,004	4,265,470	17,994,469
share of rows	10.91%	12.52%	26.28%	28.47%
Rows in conflict groups	7,018	14,140	423,290	263,772
benign vs. attack	7,018	22	170,766	263,772

The kind of conflict changes more than its size (Table III). In the 2017 corrected release almost every conflict is between two attack labels: identical port-scan probes are labelled *Infiltration – Portscan* on Thursday and *Portscan* on Friday, so the label records the scenario rather than the behaviour. In the 2018 corrected release, 263,772 rows sit in only 13 conflict groups, but they are almost all failed FTP connection attempts labelled *FTP-BruteForce – Attempted* that happen to match 60 benign flows exactly. The binary benign-versus-attack floor is therefore 7 rows in 2017 and 65 rows in 2018, far smaller than the raw conflict counts suggest, while the multi-class floor in 2017 is 4,946 rows.

TABLE III
CONFLICT GROUPS IN THE CORRECTED RELEASES

Labels in group	Groups	Rows	Floor
2017: Infiltration – Portscan / Portscan	4,368	14,118	4,939
2017: BENIGN / Portscan	7	22	7
2018: BENIGN / FTP-BruteForce – Attempted	9	263,736	60
2018: BENIGN / Infiltration – NMAP Portscan	4	36	5

F. Other observations

Rows are not stored in time order: in every corrected file, 30–45% of consecutive rows step back in time by more than a minute, and the *id* column is simply the row number at export. The 2018 day files are concatenations of per-host exports. Within-day row order and *id* therefore encode file structure, and *id* alone separates benign from attack flows in 2018 with OVL 0.33. Several attack classes are very small (for example 11 Heartbleed and 13 SQL-injection flows in 2017, and 39 SQL-injection flows in 2018), too few for per-class metrics with useful confidence intervals.

V. Original Versus Corrected Releases

A. Clocks of the original releases

Every parsed hour in both original releases lies between 1 and 12; there is no hour 0 and no hour from 13 to 23 in any of the 18 files, and no AM/PM marker. The originals therefore record local time on a 12-hour clock with the half of the day discarded. We recovered the offset to UTC by matching flows on their 5-tuple and start time (hour taken modulo 12) for every whole-hour offset from −12 to +12 on a 5% sample. For 2017, an offset of −3 h matched 72.9% of Monday's sampled

flows, against 33.6% for the next-best candidate (+9 h, the same offset on the other half of the clock). For the one 2018 original file that still contains addresses and source ports (20 February), −4 h matched 78.3%, against 35.2% for +8 h. These offsets are UTC−3 in July and UTC−4 in February, consistent with Atlantic time. An earlier version of our analysis assumed a 24-hour clock for 2018 and matched only the morning half of that day (41.9%).

B. Row-level matching

We fixed matching criteria before computing them: a day pair is usable for row-level comparison only if at least 90% of original rows match exactly one corrected row and at most 2% are ambiguous. Table IV shows that only one day pair meets them. In 2017, between 9% and 27% of each day's original flows have no counterpart, which is what removing the TCP appendix flows and re-segmenting connections should produce. Nine of the ten 2018 original files contain neither addresses nor source ports, so they cannot be matched at all. On the one qualifying pair (2017 Friday afternoon, port scan), the uniquely matched flows show 158,702 PortScan → Portscan, 13 PortScan → BENIGN, 103,009 BENIGN → BENIGN and 5 BENIGN → Botnet – Attempted transitions. This should not be generalised: unmatched flows are exactly where the corrections act.

TABLE IV
ROW-MATCHING COVERAGE, ORIGINAL TO CORRECTED

Original file	Offset	Unique 1:1	Ambig.
2017 Monday	−3 h	0.728	0.000
2017 Wednesday	−3 h	0.865	0.001
2017 Thu. afternoon (infiltration)	−3 h	0.823	0.025
2017 Fri. afternoon (port scan)	−3 h	**0.914**	0.003
2017 Fri. afternoon (DDoS)	−3 h	0.798	0.013
2018 20 Feb (has 5-tuples)	−4 h	0.783	0.001
2018 other nine days	−4 h	≤ 0.047	0.57–0.90

C. Distribution-level changes

Because rows cannot be matched at scale, the remaining comparison is between distributions, which mixes relabelling, re-segmentation, rows dropped at export and, for 2018, the truncation of seven original files at 1,048,575 rows (the spreadsheet row limit). With that caveat, the changes follow the documented fixes closely. The share of flows with no backward packet falls from 14.5% to 2.6% (2017) and from 25.5% to 1.2% (2018); the median flow duration roughly doubles in 2017 (32 ms to 64 ms); and several attack families shrink to 0.50–0.51 of their original flow count in both years (FTP and SSH brute force in 2017; SSH brute force, LOIC-HTTP, botnet and XSS in 2018). A factor of almost exactly one half is what one expects if each TCP connection previously produced a main flow and an appendix flow. The Attempted label absorbs most of some classes: only 5% of the 2017 web brute-force flows and 3% of the XSS flows remain non-Attempted, in line with the corruption rates reported in [5]. In 2018, all 298,874 FTP brute-force flows are Attempted (port closed), and 2017 gains a new class, *Infiltration – Portscan*, with 71,803 flows.

The original releases are not suitable for fine-grained temporal analysis. The 2017 originals have minute resolution (seconds on Monday only), and the 2018 originals are truncated and contain 14 timestamps in January 1970. Any comparison of temporal results between original and corrected data is possible only for 2017, at windows of 60 s or longer, after converting the 12-hour local times.

VI. TEMPORAL STRUCTURE OF THE CORRECTED DATA

We now treat the corrected releases as a time series. Flows are assigned to 2-second windows by their start time within each day file, and windows never cross files. Capture days do not overlap and are separated by night-time gaps, so there is no continuity across days.

A. Attacks are a few scripted blocks

Fig. 1 shows the typical picture: attack traffic arrives in blocks at scheduled times. Grouping attack flows of one family into episodes whenever consecutive flows are at most 300 s apart gives only 24 episodes in 2017 and 38 in 2018, and most families occur in a single episode on a single day. At a 60 s gap the counts rise to 98 and 285, mostly because slow attacks pause. No episode begins within 120 s of a file start, so each has benign history before it. Any onset metric on these data rests on a few dozen events, and per-family onset results are anecdotes.

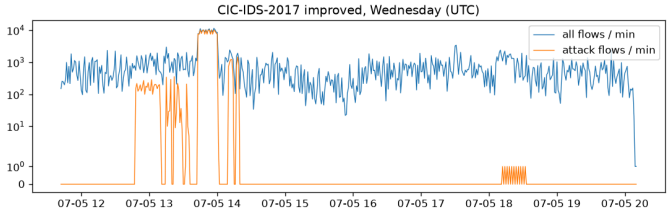


Fig. 1. Flows per minute on Wednesday, 5 July 2017 (corrected release, UTC, log scale). Attack flows (orange) occur in a few contiguous blocks, here the DoS and Heartbleed scenarios.

B. Persistence dominates next-window targets

Let $y(t)$ be 1 when at least one non-Attempted attack flow starts in window t. Over all corrected days, the next-window target is positive in 7.79% of 2017 windows (73,450 windows) and 10.35% of 2018 windows (214,422 windows, stray rows excluded). Across horizons of 1 to 5 windows, the current window is already positive for 66.9–73.0% (2017) and 90.9–93.1% (2018) of the positive targets. A predictor that copies the present state will therefore look strong at every horizon. The windows where the state changes from benign to attack number 1,548–1,895 (2017) and 1,529–2,009 (2018) per horizon, which is far more than the 24 and 38 true episode onsets: most of them are gaps inside an episode, where a slow attack starts no new flow for a few seconds. A next-window score mostly measures recurrence within an episode, not anticipation of a new one.

C. Look-ahead from long flows

A CICFlowMeter record summarises a flow over its whole life, up to the 120 s flow timeout, but carries the timestamp of

its first packet. In the corrected files, 23.9–30.7% (2017) and 22.3–38.1% (2018) of flows last longer than one 2-second window. If such a flow's duration, byte and packet totals are added to the window in which it starts, that window contains up to two minutes of the future. This is easy to do by accident, because the timestamp column looks like an event time. A forecasting setup on these data should build window features only from flows that have ended by the end of the window, or lag all flow-level features by the flow timeout.

D. Split leakage

Table V summarises candidate splits applied to all 65.3 million corrected rows, with a 130 s embargo (the flow timeout plus the longest horizon) around within-day cuts and the stray rows excluded. A random split (P0) puts an exact feature twin of 15.6% (2017) and 30.0% (2018) of test rows in the training set. In 2017, 25.8% of the test attack flows are seen in training, and all of them are port scans; every other family has none. Splitting each day by time (P1) does not work, because attacks are scheduled at fixed times of day: in 2018 the last fifth of every day contains no attack flows. Cutting each day at flow-count quantiles instead (P1b) does put attacks in the test set, but the cuts fall inside attack blocks and the embargo removes 113,522 and 716,549 attack flows. Splitting by day (P2, and forward chaining P3) avoids attack twins almost entirely, but 2017 then tests only attack types never seen in training. In 2018, three families recur on the day immediately before a test day, so they count as seen only if the final model is refit on training and validation days together; after such a refit, 45.9% of the 1 March test attacks have twins. Benign twins remain in every 2018 split (25–31% of test rows) and cannot be avoided.

TABLE V
TEST ROWS WITH AN EXACT FEATURE TWIN IN TRAINING

Split	Data	Test attacks	All seen	Attacks seen
P0 random	2017	76,200	15.6%	25.8%
P0 random	2018	530,038	30.0%	1.2%
P1 day fraction	2018	0	28.4%	—
P1b day quantile	2017	76,800	9.5%	2.1%
P1b day quantile	2018	15,184	28.8%	7.1%
P2 by day	2017	254,946	5.9%	0%
P2 by day	2018	232,584	26.6%	0.01%
P3 test 1 Mar	2018	39,834	27.5%	0.04%*

*45.9% if the model is refit on training plus validation days.

E. Single-feature shortcuts

The best single non-identifier feature reaches an OVL of about 0.09 in both corrected releases (packet-length maxima), against 0.31–0.39 in the originals. The destination port reaches 0.103 (2018) and 0.157 (2017). We cannot tell from these data whether the corrected flows are simply cleaner, or whether labelling with feature filters (the rules use ports, durations and packet lengths) makes classes more homogeneous; both would produce this result. In either case, a near-perfect per-flow score on the corrected releases says little on its own. Attempted flows are separable from attacks almost by construction (median OVL 0.044 in 2018), because some Attempted categories are defined by the absence of payload.

VII. Recommended Protocol

The measurements above lead to a small set of rules for anyone using the corrected releases, particularly for temporal tasks.

- *Name the release* (CNS 2022 corrected, post-publication files) and publish file checksums, since at least two corrected versions of CIC-IDS-2017 exist.
- *Never use as inputs* the row index, flow identifier, addresses, source port, timestamp value, labels, or file and day identity. Exclude the two header-length features, and ablate destination port, protocol and the initial-window features.
- *Sort by timestamp* within each file, never across files, and exclude the 2,609 stray rows of 23 February 2018 with the count reported.
- *Build window features from ended flows only.* Report, as an ablation, how much start-attributed totals would add.
- *Split by day in time order*, with forward-chaining folds for 2018, leave-one-day-out, and cross-dataset transfer as robustness checks. Score recurring and unseen families separately and report a de-duplicated test score alongside the raw one. Use random splits only to show their optimism.
- *Report persistence.* The fair baseline carries forward the output of a current-window detector; copying the true current label is an upper bound, not a baseline. Score benign-to-attack transitions and true episode onsets separately from all windows.
- *Report the floors* of Table III next to per-class results, state the Attempted policy explicitly, and give metrics with operational meaning: precision–recall rather than ROC for rare classes [20], false alerts per hour [21], and bootstrap intervals over days or episodes rather than over autocorrelated windows.

VIII. Threats to Validity

The label check covers one direction only. We did not re-run the labelling code, so flows that match a rule but were left benign are not counted. The UTC interpretation of the corrected timestamps rests on the authors' documentation and on internal consistency; the recovered offsets of the originals support it but do not prove it. The corrected export dropped rows with infinite rates, and we cannot tell how many. Duplicate detection uses 64-bit row hashes; the expected number of collisions among about 45 million distinct 2018 vectors is around 5×10^{-5}, and the 2017 figures were confirmed with a second hash. OVL is computed one feature at a time on coarse bins and does not capture shortcuts that combine features. Finally, the literature screening was done by one reviewer at title and abstract level, and several closely related works, including SENTINEL [16] and the IDS2Eval paper [18], could be read only as abstracts or artifact records. Our statements about prior work should be read with that limit in mind.

IX. Conclusion

The CNS 2022 corrections removed most of what was wrong with CIC-IDS-2017 and CSE-CIC-IDS-2018, and their labels follow the published logic exactly. They are not free of defects. Two header-length features overflow, a day of stray flows remains in one 2018 file, and roughly an eighth of 2017 and more than a quarter of 2018 rows are duplicates once identifiers are removed. The original releases, still widely used, record time on a 12-hour clock without AM/PM. For temporal work the larger issue is structural rather than a defect: attacks come as a few dozen scripted blocks, so persistence explains most next-window positives, long flows leak future information into time windows, and the split decides which attacks can be "predicted" at all. Results on these data are informative only when they are compared against persistence, scored on true onsets, built from ended flows and split by day. We release the audit code and all tables so that these numbers can be checked and extended.

Data and Code Availability

The audit code, configuration, result tables and reports are available at https://github.com/whateveryoucanthinkof-eng/kira-final (folder "research paper"). The datasets themselves are not redistributed; SHA-256 checksums of every file audited are included.

Acknowledgments

The author thanks the authors of [5] for releasing their corrected datasets and labelling code, which made this audit possible. An AI assistant (Claude, Anthropic) was used to help write and run the analysis code and to draft and edit the text. All numbers come from the released scripts and tables, and the author takes responsibility for the content.

References

[1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.

[2] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: the CICIDS2017 case study," in *Proc. IEEE Security and Privacy Workshops (SPW)*, 2021, pp. 7–12, doi: 10.1109/SPW53761.2021.00009.

[3] A. Rosay, E. Cheval, F. Carlier, and P. Leroux, "Network intrusion detection: a comprehensive analysis of CIC-IDS2017," in *Proc. 8th Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2022, pp. 25–36.

[4] M. Lanvin, P.-F. Gimenez, Y. Han, F. Majorczyk, L. Mé, and É. Totel, "Errors in the CICIDS2017 dataset and the significant differences in detection performances it makes," in *Risks and Security of Internet and Systems (CRiSIS 2022)*, LNCS 13857, Springer, 2023, pp. 18–33.

[5] L. Liu, G. Engelen, T. Lynar, D. Essam, and W. Joosen, "Error prevalence in NIDS datasets: a case study on CIC-IDS-2017 and CSE-CIC-IDS-2018," in *Proc. IEEE Conf. Communications and Network Security (CNS)*, 2022, pp. 254–263, doi: 10.1109/CNS56114.2022.9947235.

[6] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *Proc. 31st USENIX Security Symp.*, 2022, pp. 3971–3988.

[7] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: formulation, detection, and avoidance," *ACM Trans. Knowledge Discovery from Data*, vol. 6, no. 4, art. 15, 2012.

[8] R. Geirhos, J.-H. Jacobsen, C. Michaelis, R. Zemel, W. Brendel, M. Bethge, and F. A. Wichmann, "Shortcut learning in deep neural networks," *Nature Machine Intelligence*, vol. 2, pp. 665–673, 2020.

[9] L. D'hooge, M. Verkerken, B. Volckaert, T. Wauters, and F. De Turck, "Establishing the contaminating effect of metadata feature inclusion in machine-learned network intrusion detection models," in *Proc. DIMVA*, LNCS 13358, Springer, 2022, pp. 23–41.

[10] M. Cantone, C. Marrocco, and A. Bria, "On the cross-dataset generalization of machine learning for network intrusion detection," arXiv:2402.10974, 2024.

[11] R. Flood, G. Engelen, D. Aspinall, and L. Desmet, "Bad design smells in benchmark NIDS datasets," in *Proc. IEEE European Symp. Security and Privacy (EuroS&P)*, 2024.

[12] Z. Moczkodan and H. Ragab, "Do transformers actually help intrusion detection? A temporal sequence evaluation on CIC-IDS2017," arXiv:2606.11098, 2026.

[13] M. Youssef, "Stream assembly is an uncontrolled treatment in streaming intrusion-detection benchmarks," arXiv:2605.24696, 2026.

[14] M. Husák, J. Komárková, E. Bou-Harb, and P. Čeleda, "Survey of attack projection, prediction, and forecasting in cyber security," *IEEE Commun. Surveys & Tutorials*, vol. 21, no. 1, pp. 640–660, 2019.

[15] Y. Ahmed, M. A. Azad, and T. Asyhari, "Rapid forecasting of cyber events using machine learning-enabled features," *Information*, vol. 15, no. 1, art. 36, 2024.

[16] Shahane et al., "SENTINEL: an LSTM world model for proactive network attack forecasting from network traffic telemetry," preprint, Zenodo, 2026, doi: 10.5281/zenodo.22212693.

[17] X. Wang and R. Stadler, "A modular framework for rapidly building intrusion predictors," arXiv:2511.23000, 2025.

[18] "IDS2Eval: a falsification-oriented framework for auditing machine-learning intrusion detection benchmarks," artifacts, Zenodo, 2026, doi: 10.5281/zenodo.23301257.

[19] M. Raasveldt and H. Mühleisen, "DuckDB: an embeddable analytical database," in *Proc. ACM SIGMOD*, 2019, pp. 1981–1984.

[20] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLOS ONE*, vol. 10, no. 3, e0118432, 2015.

[21] S. Axelsson, "The base-rate fallacy and the difficulty of intrusion detection," *ACM Trans. Information and System Security*, vol. 3, no. 3, pp. 186–205, 2000.